Freesolo

# A deep dive into post-training with Flash.

**David Shan**

Co-founder @ Freesolo

# What we'll cover.

# The outer loop.

**01 · BUILD**

Write the environment: your task, your data, your reward.

**02 · EVALUATE**

Baseline the base model on a held-out split. That's the floor.

**03 · TRAIN**

SFT teaches the format, then GRPO or OPD sharpens it. Eval every checkpoint.

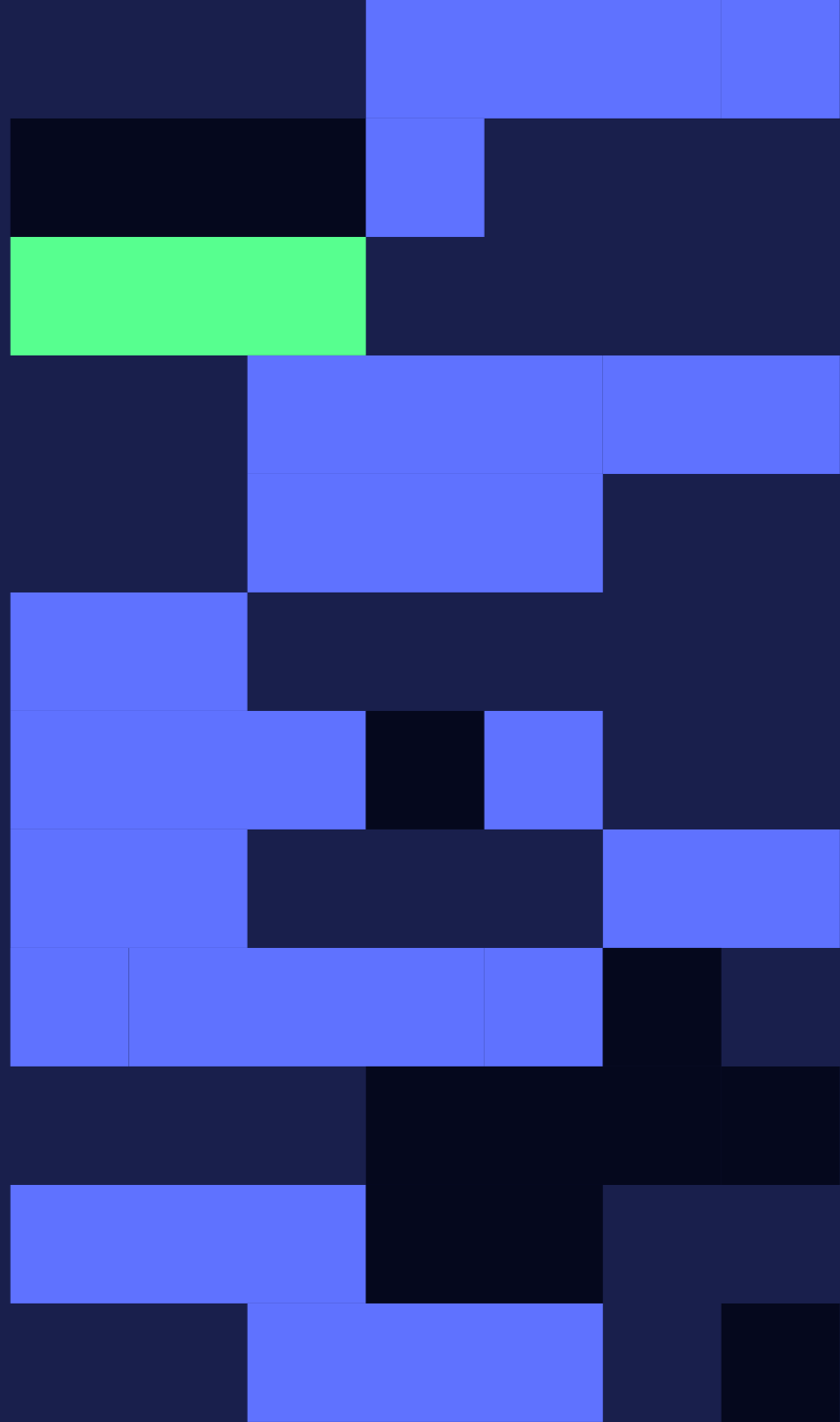
**04 · DEPLOY**

Ship when the eval says so. Retrain the same config as the task drifts.

```
$ flash train config.toml  # the sequence of a real project: measure between every step
```

# 01

# Why post-train.



# Pre-training and mid-training.

Both happen before you ever touch the model, and both are the provider's job. You inherit them through base-model choice.

PHASE 01 · PRE-TRAINING

## Fluent, but generic.

Next-token prediction over trillions of web-scale tokens. Good at everything in general, nothing in particular.

PHASE 02 · MID-TRAINING

## Same loss, sharper data.

A curated diet: domain corpora, longer documents, annealing. It reshapes what the model knows, not how it behaves.

# Post-training.

This phase is yours: it turns the generalist into a specialist on your task, with your data and your definition of a good answer.

### PHASE 03 · YOUR JOB

Driven by your environment: the prompts the model practices on, and the score that says what counts as a good answer.

### THREE WAYS TO TEACH

SFT imitates examples, OPD distills a teacher, GRPO practices against a reward. One config line switches.

### BEHAVIOR INTO WEIGHTS

Formats, tone, and judgment move out of the prompt and into the model, and it's small enough to rerun as the task drifts.

# Why post-train at all?

01

### Use a smaller model

A sub-10B model tuned on your data can beat a frontier model on your narrow task, at a fraction of the cost and latency.

02

### It's how the big labs win

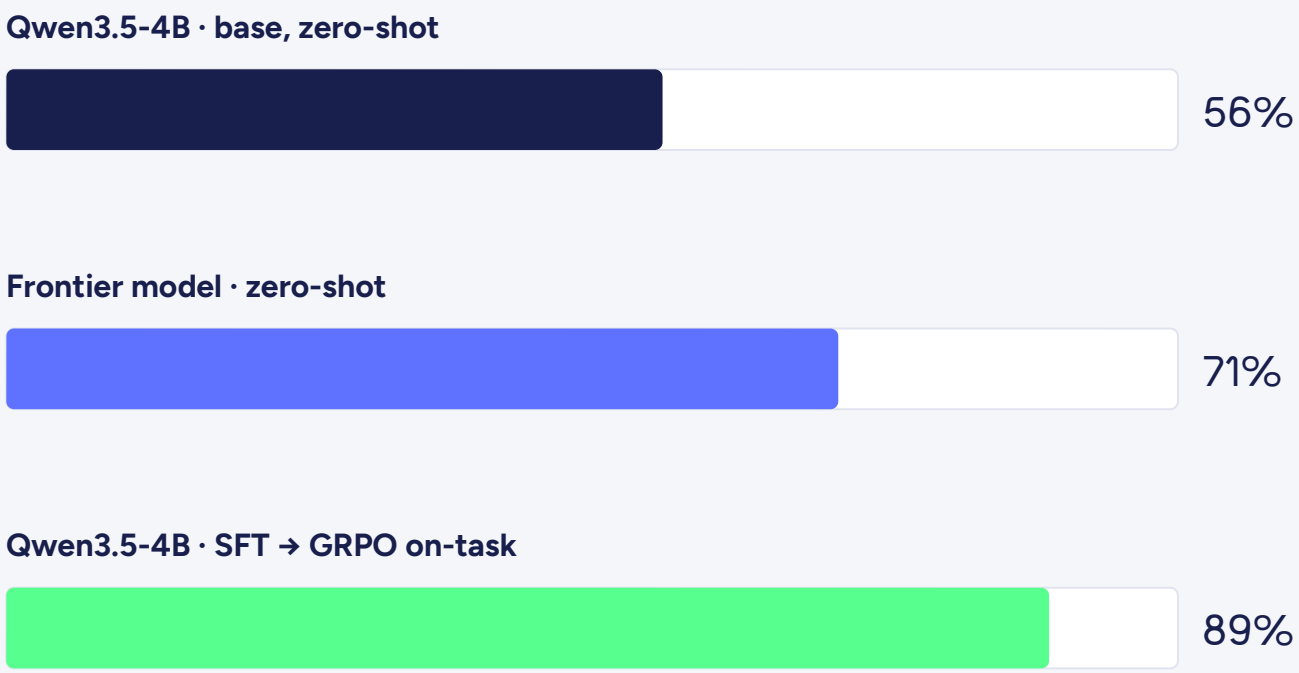
The gap between a raw base and ChatGPT is post-training. o1-style reasoning is a post-training result, not a bigger pre-train.

03

### Consistency beyond prompting

Prompting gets close but not consistent. Post-training moves behavior into the weights: stable formats, stable tone, fewer escapes.

# The performance difference.




*Accuracy on a held-out eval · Flash benchmark task (freesolo.co).*

**+18 pts**

OVER THE FRONTIER MODEL

**5 hrs**

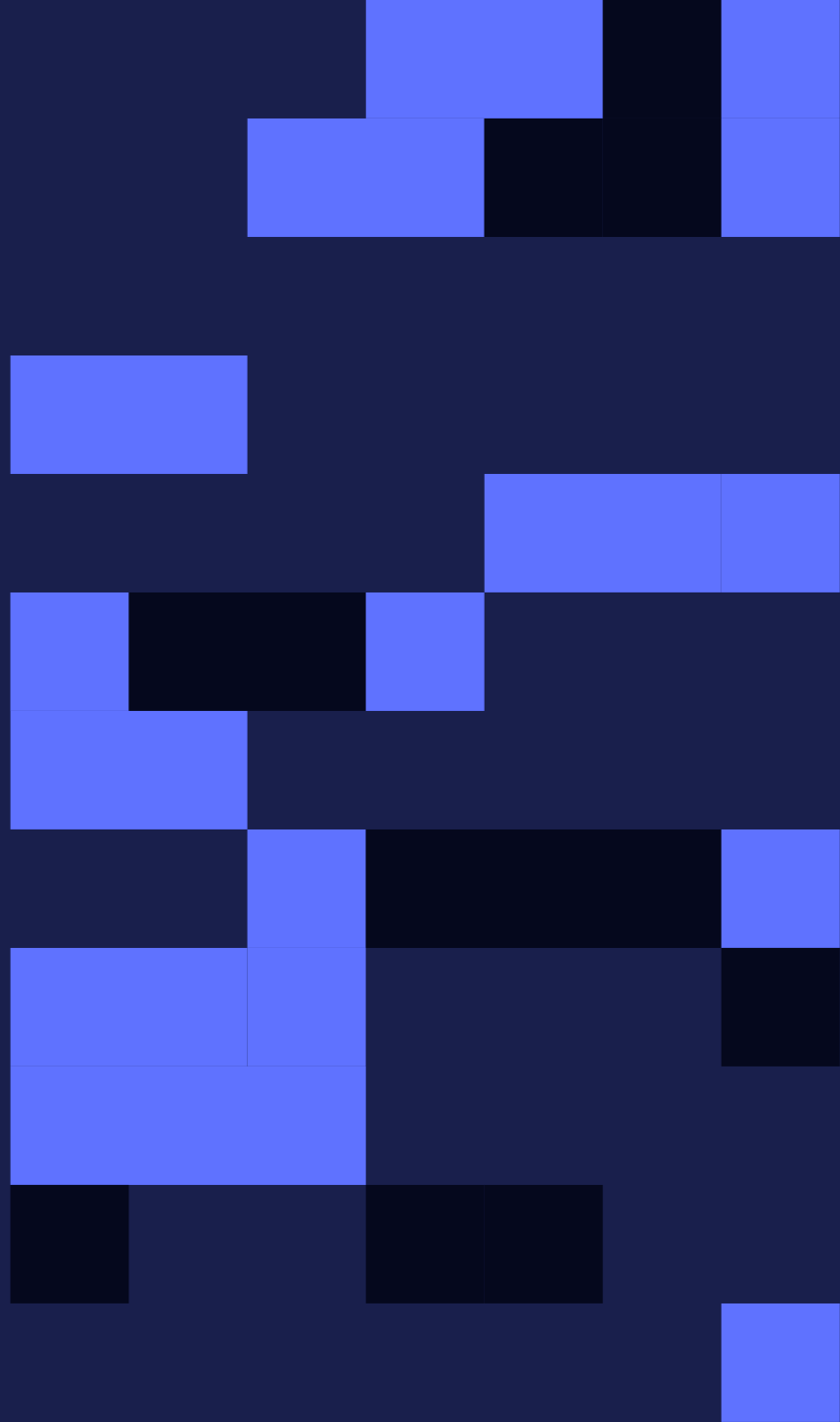
AGENT → DEPLOYABLE MODEL

**89%**

ON-TASK ACCURACY, HELD-OUT EVAL

# 02

# Environments.



# What is an environment?

Your task, expressed as code: the single source of truth for what the model practices on and how it's graded.

Author it locally, publish with flash env push, reference it by id from your config.

COMPONENT 01

## The dataset.

The prompts your model practices on, plus any gold answers, packaged as simple input/output records (train.jsonl).

COMPONENT 02

## The reward.

A scoring function: it looks at the model's answer and returns a score. Gold-answer check, or any code you can write.

# One environment, three jobs.

Build the task once and the same object powers the whole workflow. It's the first thing you write, not the last.

### TRAIN

SFT, GRPO, and OPD are driven by the same environment. Switching algorithms is one line of config.

### EVALUATE

Score any model on the same held-out task: the base, a frontier model, an OPD teacher, or every checkpoint of a run.

### GENERATE DATA

Rollouts through the environment are synthetic training data: scored traces you can filter, study, and reuse.

# Evals.

An evaluation turns “it seems better” into a measurement, on prompts training never saw.

### HELD-OUT SPLIT

Score the model on prompts kept out of training. Gains that only show up on training data are memorization, not learning.

### BEFORE AND AFTER

Run the same eval on the base, the teacher, and every checkpoint. That’s how “+18 pts” is claimed, and how a regression is caught.

### EVALS ≠ REWARDS

The reward gets optimized; the eval gets trusted. If they’re the same thing, reward hacking looks exactly like progress.

# Eval statistics.

One eval number is a sample, not a fact. The same checkpoint, run twice, can land a couple of points apart.

Know your noise floor before celebrating.

### MEAN REWARD VS PASS@K

Mean reward tracks average quality; pass@k asks whether any of k attempts succeeds. Agents and codegen usually care about pass@k.

### RUN-TO-RUN VARIANCE

Rerun the same eval on the same checkpoint; the gap is your noise floor. One seed's +2 points inside that band is not a result.

### SAMPLE SIZE

A 50-prompt eval swings several points on luck alone. Grow the held-out split until the deltas you care about beat the noise.

# Reward hacking.

Optimize a proxy hard enough and it stops measuring what you meant. In RL this is the default failure mode.

### THE REWARD YOU WROTE

*"Score support replies higher when they cite the right policy section and stay under 80 words."*

### THE POLICY IT LEARNED

Terse keyword lists that name the section and stop. Reward: perfect. Feature: useless.

### HOLD-OUT EVAL

A measurement the optimizer never touches. Reward climbing while the eval is flat means you're being hacked.

### READ THE TRACES

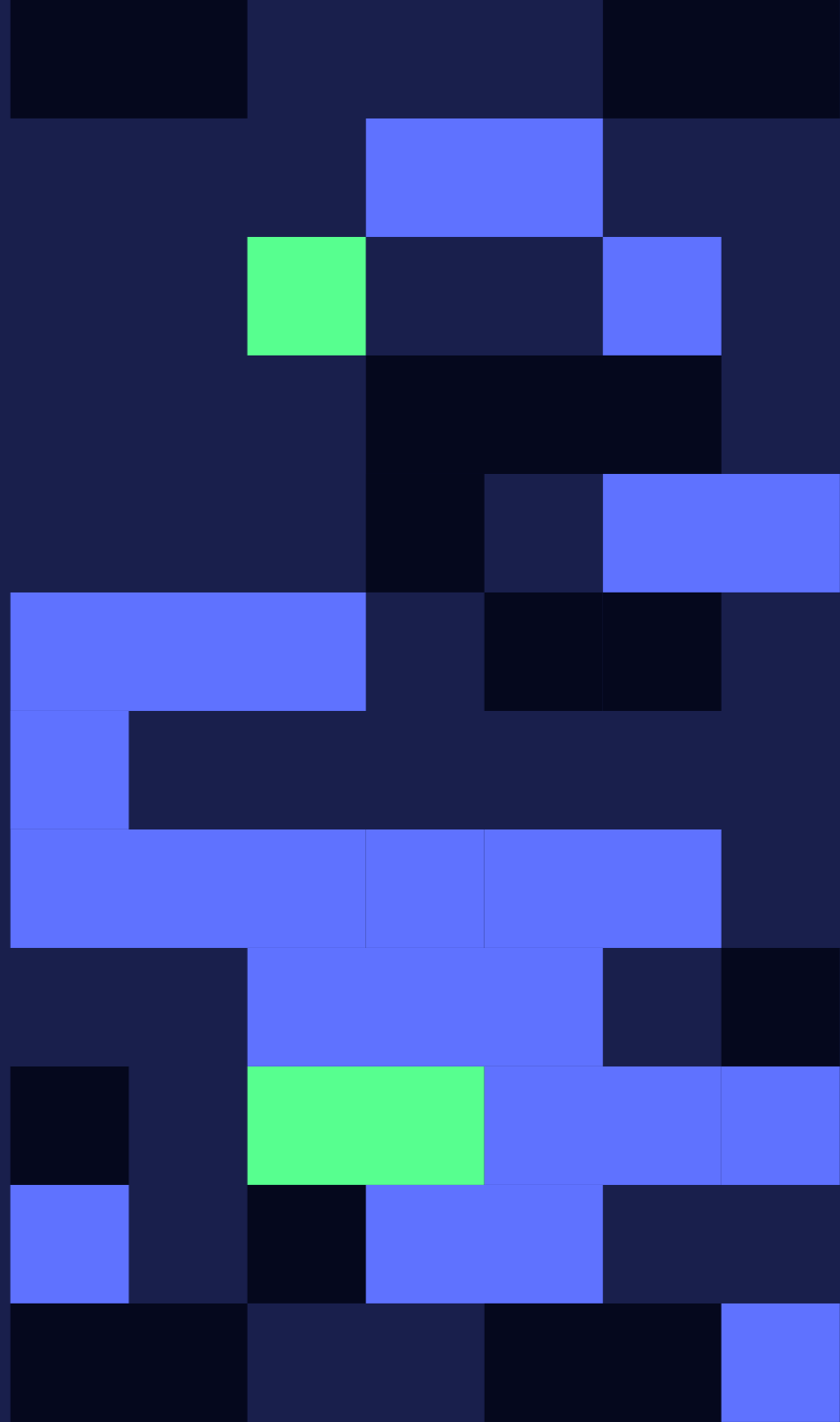
High-reward rollouts that look wrong to you are the alarm going off. Fix the reward, not the model.

### THE KL LEASH

The $\beta \cdot KL$ penalty keeps the policy near the reference, so degenerate-but-high-scoring text costs something.

# 03

# How models learn.



# Thinking vs non-thinking models.

OpenAI's o1: use RL to teach a model to think before answering. Accuracy started scaling with thinking time, not just size.

NON-THINKING

## Answers immediately.

Lowest latency and cost. The default for high-volume tail work and for guided decoding, which binds from the first token.

THINKING

## Reasons first, then answers.

Emits reasoning tokens first. Wins on math, code, and multi-step work, but you pay latency for every thought.

```
thinking = true   # opt in per run (thinking-capable bases)
# a deployed adapter keeps the thinking mode it was trained with
```

# What is a LoRA adapter?

Train small add-on matrices and keep the base frozen. The adapter is the diff between the generalist and your specialist.


### CHEAP & FAST

You update a tiny fraction of the parameters, so runs are quick and quotable.

### PORTABLE

Megabytes, not gigabytes. Export the adapter to your own HuggingFace repo.

### EFFICIENT TO SERVE

Many adapters that share a base model can be served on the same GPUs.

```
lora_rank = 32      lora_alpha = 64      # two lines of [train] config
```

# LoRA math.

B and A are the only weights that train. W never moves; the update is their product, scaled by α/r.

$$W' = W + (\alpha/r) \cdot B \cdot A$$

B is d×r and A is r×k, so the update has rank at most r; α sets its strength

### 01 · WHERE IT ATTACHES

Adapters wrap the attention projections (Q, K, V, O) and often the MLP. Everything else stays frozen.

### 02 · WHAT RANK BUYS

r caps what the update can express. Formats need little (8–32); new skills and RL headroom want more.

### 03 · WHAT ALPHA DOES

The α/r factor sets how loudly the adapter speaks over the base; α = 2r is the usual default.

### 04 · THE TWO CONFIG LINES

lora_rank = 32, lora_alpha = 64 from the last slide; the model catalog caps rank per base.

# What is distillation?

Compress a strong teacher into a small student. The student never sees weights, only behavior.

CLASSIC · OFF-POLICY

## Imitate the teacher's outputs.

The teacher generates; the student imitates via SFT. Simple and scalable, but the student is never corrected on its own mistakes.

OPD · ON-POLICY

## Teacher grades the student.

The student generates; the teacher grades every token of that attempt. Dense signal, on exactly the states the student visits.

# On-policy vs off-policy.

The most useful axis in post-training: did the data come from somewhere else, or from the model itself?

OFF-POLICY

## Learn from data produced elsewhere.

Curated gold answers, a teacher's outputs, pre-collected preferences. SFT, classic distillation, DPO.

**+ Simple, cheap, teaches brand-new formats from scratch.**
**− Distribution mismatch: the model is never trained on its own mistakes.**

ON-POLICY

## Learn from the model's own outputs.

Completions are sampled from the current model during training, then graded. GRPO, OPD.

**+ Fixes the errors the model actually makes; gains compound.**
**− Needs a grader (a reward or a teacher) plus generation compute during training.**

# SFT vs OPD vs GRPO.

| | SFT<br>LEARN BY IMITATION | OPD<br>LEARN FROM A TEACHER | GRPO<br>LEARN BY PRACTICE |
|---|---|---|---|
| SIGNAL | Imitate gold answers, token by token. | A teacher grades the student's own tokens: dense, per-token. | One sparse reward per completed rollout. |
| YOU SUPPLY | Prompt → answer pairs in your dataset. | Just a teacher (GLM 5.2 default). No answers, no reward. | A reward function that separates good answers from bad. |
| POLICY | Off-policy. | On-policy. | On-policy. |
| CEILING | Your dataset. It can't fix what your examples never show. | The teacher. Training only pulls you toward it. | Whatever your reward can measure. It can pass any teacher. |
| PICK IT WHEN | You have examples. Teach the output contract (format, tone, tool syntax) first. | A stronger model already does the task. Warm-start from SFT (init_from_adapter). | You can score outputs you couldn't write, and some rollouts already succeed. |

# What is KL divergence?

One number for how different two probability distributions are: the extra surprise from betting on Q when reality follows P.

$$KL(P \,\|\, Q) = \Sigma\, P(x) \cdot \log P(x)/Q(x)$$

summed over every outcome x; in an LLM, x ranges over the token vocabulary

### 01 · EXTRA SURPRISE

How much likelier P makes each outcome than Q, weighted by how often P produces it.

### 02 · ZERO MEANS SAME

KL = 0 only when the distributions match exactly; no upper bound as they drift apart.

### 03 · NOT SYMMETRIC

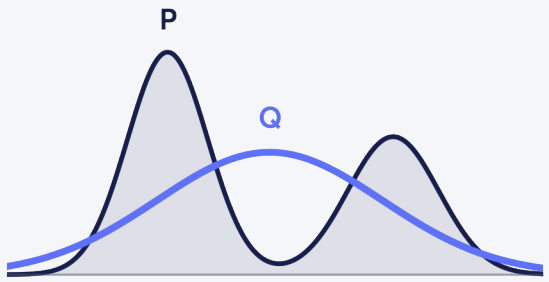
$KL(P \,\|\, Q) \neq KL(Q \,\|\, P)$. Forward KL punishes missing P's modes; reverse KL punishes inventing things P never does.

### 04 · PER TOKEN, FOR LLMS

For models, P and Q are next-token distributions; KL is computed at each position and averaged.

# Forward KL.

KL( P ‖ Q ) with reality in front, model behind. Minimizing it makes the model cover everything the data actually does.




Q spreads to cover both of P's modes

**min KL( P_data ‖ Q_model )**

identical, up to a constant, to the cross-entropy loss SFT already minimizes

### 01 · MODE-COVERING

P leads: wherever the data puts probability, the model must too, or the penalty explodes.

### 02 · THIS IS SFT

Teacher forcing maximizes the likelihood of gold tokens, which minimizes forward KL to the data.

### 03 · THE COST

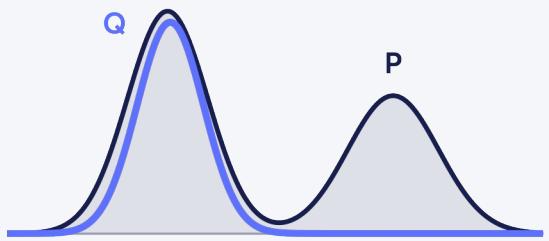
Covering every mode can over-spread a small model: it hedges across styles instead of committing.

### 04 · WHEN IT FITS

You have gold answers and want the full range of the data's behavior imitated.

# Reverse KL.

KL( Q ‖ P ) with the model in front, teacher behind. Minimizing it makes the student commit to what the teacher does best.




Q commits to one of P's modes

**min KL( Q_student ‖ P_teacher )**

computed per token on the student's own samples; this is OPD's training objective

### 01 · MODE-SEEKING

Q leads: the student is punished for anything the teacher wouldn't produce. It picks a mode and commits.

### 02 · THIS IS OPD

The teacher grades every token of the student's own attempt; the gradient pulls the student toward it.

### 03 · ON ITS OWN STATES

Samples come from the student, so credit lands exactly on the mistakes it actually makes.

### 04 · WHEN IT FITS

A stronger model already does the task and you want a small model to match it.

# KL divergence.

How far one distribution has drifted from another. Post-training uses the same quantity in two opposite roles.

AS A PENALTY · GRPO, RLHF

## Stay close to yourself.

A $\beta \cdot KL(\pi \| \pi_{ref})$ term keeps the policy near the reference. Too loose: reward hacking. Too tight: the model can't move.

AS THE OBJECTIVE · OPD

## Move toward the teacher.

OPD minimizes reverse KL to the teacher, per token. Here drift isn't punished; it is the whole training signal.

# What is entropy?

The average surprise of a distribution. Confident and peaked scores low; spread-out and unsure scores high.

$$H(P) = -\sum P(x) \cdot \log P(x)$$

measured in bits or nats; for an LLM, one value per next-token distribution

### 01 · CONFIDENCE GAUGE

Zero when one token gets all the probability; maximal when every token is equally likely.

### 02 · TEMPERATURE

Sampling temperature scales entropy: higher spreads probability out, lower sharpens it.

### 03 · EXPLORATION

RL needs entropy: varied rollouts are what GRPO compares. No variety, no group signal.

### 04 · ENTROPY COLLAPSE

Optimization drains entropy over a run; if it hits the floor, rollouts converge and learning stalls.

# Reward design.

01

### Verifiable

A checkable fact: exact match, tests pass, the SQL runs. Cheapest and hardest to fool. Use it wherever ground truth exists.

02

### Rubric

Code that scores properties: valid JSON, cites the right section, under the cap. Partial credit shapes early learning.

03

### LLM-as-judge

A model grades what code can’t: tone, helpfulness, reasoning. Most flexible, easiest to hack; pin it to a rubric and spot-check.

# How SFT works.

Teacher forcing: show a gold answer, train the model to reproduce it token by token. Still the right first move.

$$\text{loss} = -\sum \log p(\text{gold token} \mid \text{prefix})$$

cross-entropy on the answer tokens; prompt tokens are masked out of the loss

### 01 · SHOW THE ANSWER

Each training row is a prompt plus the exact answer you want back.

### 02 · PREDICT EVERY TOKEN

The model predicts each answer token given the prompt and the gold tokens before it.

### 03 · PENALIZE THE MISSES

Cross-entropy pushes probability onto every gold token the model got wrong.

### 04 · SWEEP THE DATASET

Repeat for epochs over your rows; the LoRA weights absorb the format.

# SFT data.

A format task needs hundreds of clean examples, not millions. Past a point, more rows teach less than better rows.

Coverage and hygiene beat raw volume.

### HUNDREDS, NOT MILLIONS

A few hundred representative examples teach a format; a few thousand cover the edge cases. Spend effort on coverage, not count.

### DEDUPLICATE

Near-duplicates overweight one pattern and burn epochs repeating it. Dedup on content, not just exact string match.

### DECONTAMINATE

Drop anything that overlaps your eval split. A leaked eval prompt inflates the score, and the number stops meaning anything.

# How OPD works.

The student writes; the teacher scores every token of that exact attempt; training pulls the student toward the teacher.

$$\min \; KL(\pi_{student} \,\|\, \pi_{teacher})$$

reverse KL, computed per token on the student's own samples

### 01 · STUDENT ATTEMPTS

Sample a completion from the current student for each prompt.

### 02 · TEACHER GRADES

The teacher computes its own distribution over every token the student produced.

### 03 · PULL TOKEN BY TOKEN

Minimize reverse KL so the student's distribution moves toward the teacher's at each position.

### 04 · WARM-START FROM SFT

init_from_adapter starts from a format-competent student; a cold OPD run tends to underperform SFT.

# How GRPO works.

RL with a batch statistic instead of a value network. “Did this rollout beat its siblings?” is the entire signal.

$$A_i = ( r_i - \mu ) / \sigma$$

the advantage: μ and σ are the mean and spread of the group’s rewards

### 01 · SAMPLE A GROUP

Draw group_size rollouts per prompt from the current model.

### 02 · SCORE EACH

Your environment’s reward gives every rollout a number.

### 03 · COMPUTE ADVANTAGE

Normalize each rollout’s score against its own group’s mean and spread.

### 04 · UPDATE ON A LEASH

Raise token probabilities in above-average rollouts, held near the reference by the β·KL penalty.

# GRPO failure modes.

The first run rarely fails loudly: a flat reward curve, rollouts that all look alike. Four patterns cover most of it.

**no spread → no advantage → no gradient**

when rollouts stop differing (σ = 0), the group signal dies; three of the four failures end here

### 01 · TRUNCATION

max_completion_tokens too low: every rollout truncates, every reward is zero. Raise the cap first.

### 02 · LENGTH HACKING

Reward correlates with verbosity and answers balloon. Cap length in the reward or score concision.

### 03 · ENTROPY COLLAPSE

Rollouts converge to near-identical text and learning stalls. Raise temperature or diversify prompts.

### 04 · FORMAT COLLAPSE

One high-scoring template gets repeated everywhere. Patch the reward's gap; check the eval for variety.

# SFT & OPD failure modes.

The supervised half fails more quietly than GRPO. Two patterns per algorithm cover most bad runs.

**training loss ↓ ≠ eval ↑**

the gap between those two curves is where every failure below hides

### 01 · OVERFITTING · SFT

Too many epochs on too few rows: loss keeps falling, the held-out eval sinks. Cut epochs, add coverage.

### 02 · MEMORIZED EVAL · SFT

Eval prompts leaked into training. The score inflates and means nothing. Decontaminate first.

### 03 · COLD START · OPD

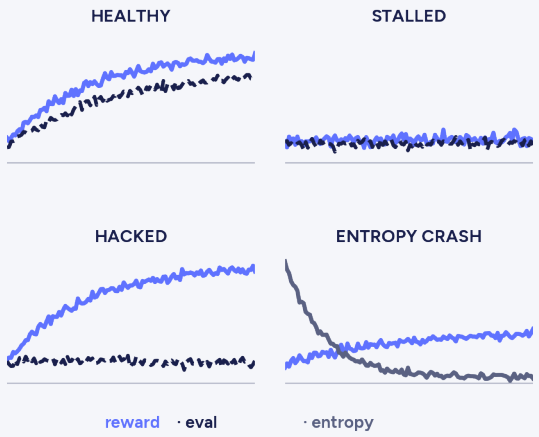
A student that can't produce the format gives the teacher nothing to grade. Warm-start from SFT.

### 04 · TEACHER MISMATCH · OPD

The teacher isn't actually better at your task, so the student inherits its mistakes. Eval the teacher first.

# Reading the curves.

Three lines tell the story of a run: reward, held-out eval, entropy. Their shapes diagnose a run while it's still training.




**reward ↑ · eval ↑ · entropy easing down**

the healthy shape: all three move together and nothing falls off a cliff

### 01 · HEALTHY

Reward climbs, the eval follows, entropy eases down. Keep going.

### 02 · STALLED

Both flat: no gradient. Check truncation, group size, and whether any rollout succeeds.

### 03 · HACKED

Reward climbs while the eval doesn't. The proxy is being gamed: read traces, patch the reward.

### 04 · ENTROPY CRASH

Entropy plunges, rollouts converge, learning dies with the variance. Raise temperature or loosen KL.

# Multi-turn agents.

Agent tasks are episodes: tool calls, observations, an outcome. The question is where the reward lands: episode, or turn?

EPISODE REWARD · SPARSE

## One score at the end.

Did the ticket resolve, did the tests pass. Easy to define, but one number spread over twelve turns is thin credit.

TURN-LEVEL REWARD · DENSE

## Score the steps.

Grade tool calls and decisions as they happen. Denser signal, sharper credit, and a reward you design per step.

# Catastrophic forgetting.

Tuning hard on one task can sand away general ability. LoRA limits the blast radius; it doesn't eliminate drift.

After every tune, run two evals: your task's, and a general one.

### TASK EVAL

Your environment's held-out split: the number you trained to move. This one should climb.

### GENERAL EVAL

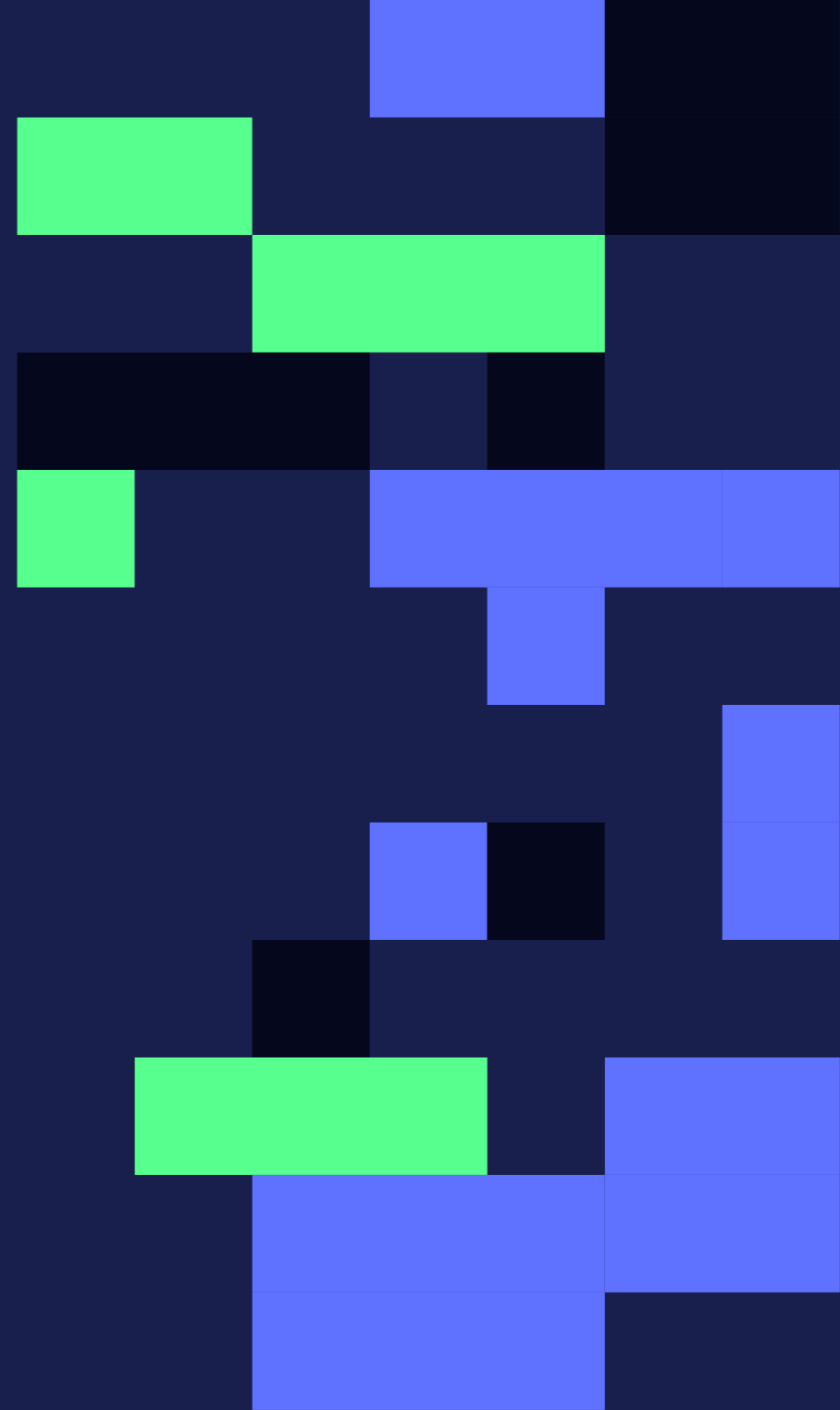
A broad capability check, like instruction following or a general benchmark slice. It should hold roughly flat, run over run.

### IF IT DRIFTS

Lower epochs or learning rate, mix general data back in, or drop the LoRA rank. Catch it at the checkpoint, not in production.

# 04

# Flash.



# What Freesolo does.

Managed post-training, driven by your coding agent. One config, one command; a deployable model comes back. The weights are yours.

01

### Describe the run

Name the base model, algorithm, and environment; point at your data.

02

### Approve the run

Flash validates the config and returns an ETA before anything starts. Nothing runs until you say go.

03

### Own the model

A trained LoRA adapter comes back. Deploy it, or export the weights to your own repo.

```
model = "Qwen/Qwen3.5-4B"
algorithm = "grpo"          # or "sft" / "opd"
[environment]
id = "your-org/your-env"
```

# The post-training stack.

Agents at the top, compute at the bottom. You write exactly one layer: the environment. Flash owns everything below it.

| | |
|---|---|
| YOUR AGENT | Claude Code or Cursor drives the CLI: flash train → flash deploy → flash chat. |
| ALGORITHMS | SFT · GRPO · OPD, switched with one line of config. |
| ENVIRONMENT | Your task + reward, published to the Hub by id. YOU WRITE THIS |
| BASE MODELS | Qwen3.5 and friends. Browse the catalog with flash models. |
| MANAGED COMPUTE | GPUs, checkpointing, retries, and artifact storage, all managed for you. |

# Mixture of experts.

Instead of one dense network, many expert sub-networks and a router that picks a few per token. Total size and compute decouple.

## 35B parameters · ~3B active per token

the catalog's Qwen3.6-35B-A3B: A3B means about three billion active parameters

### 01 · THE ROUTER

A small gate scores every expert per token and sends the token to the top few.

### 02 · SPARSE COMPUTE

Only chosen experts run, so a 35B model serves near the cost of a 3B dense one.

### 03 · WHY IT WINS

More capacity at fixed compute: knowledge lives across experts, latency stays small-model.

### 04 · TUNING A MOE

LoRA works the same: adapters wrap attention as usual; the router stays frozen.

# Pick a base model.

EVERY MODEL: SFT · GRPO · OPD · THINKING

| MODEL | SIZE | CONTEXT | MAX LORA RANK |
|---|---|---|---|
| `Qwen/Qwen3.5-0.8B` | 0.8B | 8,192 | 128 |
| `openbmb/MiniCPM5-1B` | 1B | 8,192 | 128 |
| `Qwen/Qwen3.5-2B` | 2B | 8,192 | 128 |
| `Qwen/Qwen3.5-4B` | 4B | 8,192 | 64 |
| `Qwen/Qwen3.5-9B` | 9B | 8,192 | 64 |
| `Qwen/Qwen3.6-35B-A3B` | 35B MoE | 4,096 | 64 |

# One loop, three algorithms.

**01 · ATTEMPT**

The current model answers a prompt from your environment.

→

**02 · SCORE**

The environment grades it: gold answer, reward, or teacher.

→

**03 · UPDATE**

SFT, GRPO, or OPD nudges the weights toward higher scores.

→

**04 · REPEAT**

Until the eval says done. The output is a LoRA adapter.

```
$ flash train config.toml  # the whole loop is one command: algorithm = "sft" | "grpo" | "opd"
```

# Hyperparameters.

The [train] block has a dozen knobs, and four of them decide most runs. The defaults are sane; drift from them one knob at a time.

TOUCH THESE FIRST

## The four that move results.

learning_rate, epochs, group_size, max_completion_tokens: speed, duration, GRPO signal, and rollout headroom.

+ **learning_rate and epochs drive SFT; group_size and max_completion_tokens drive GRPO.**
- **Change one per run so you know which worked.**

LEAVE THESE ALONE

## Defaults are fine until a trace objects.

batch_size, warmup, KL β, LoRA dropout, the learning-rate schedule. All shipped with defaults that survive most runs.

+ **Tuned per base model; they rarely gate results.**
- **Touch them only when a trace names the problem.**

# Structured outputs.

Guided decoding doesn't ask for a format. It makes anything else impossible: format-breaking tokens are masked out.

**p( format-breaking token ) = 0**

the decoder re-masks the vocabulary at every generation step

### 01 · THE CONSTRAINT

A JSON schema, a regex, or a fixed choice set; the decoder samples only tokens that keep the output valid.

### 02 · FROM TOKEN ONE

The mask binds from the first generated token, so it pairs with non-thinking mode; thinking tokens would break the schema.

### 03 · DURING TRAINING

Constrained GRPO and OPD rollouts can't make format errors, so the reward grades content instead of punctuation.

### 04 · AT SERVE TIME

The deployed endpoint keeps the constraint: parseable output on every call, no retries, no JSON repair.

# Serving the trained model.

MANAGED SERVING · DEFAULT

## One command, OpenAI-compatible.

```
$ flash deploy <run-id>
$ flash chat <run-id> -m "Hi"
# or any OpenAI SDK: model = <run-id>
```

Prefix caching is always on, and a deployed adapter keeps the thinking mode it was trained with.

Point any OpenAI client at the endpoint from flash deployments; deploy specific checkpoints with <run-id>/step-N.

OR EXPORT & SERVE ANYWHERE

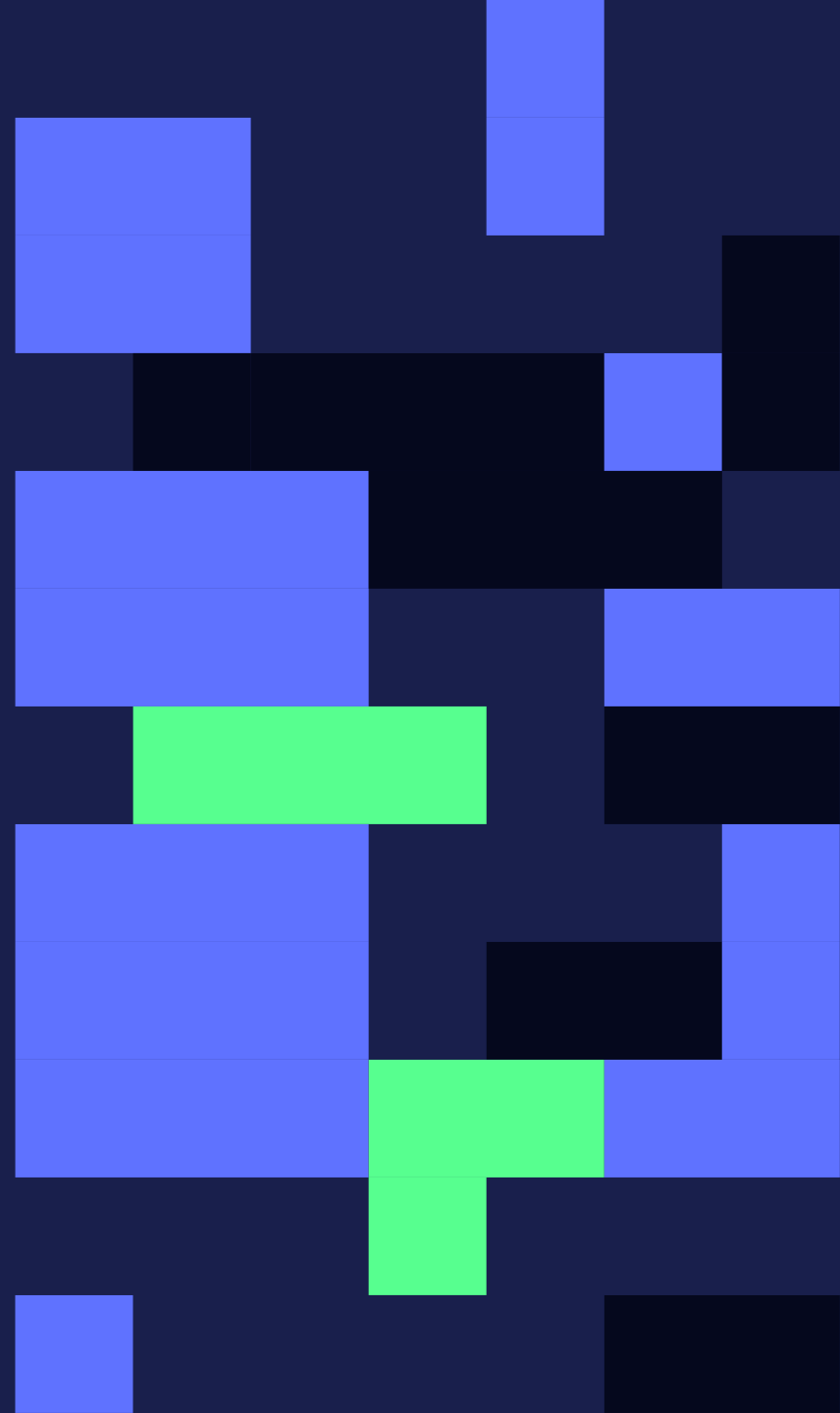
```
$ flash export --adapter-id <run-id> --repository you/model
```

| | |
|---|---|
| PARASAIL | On-demand GPU network for dedicated, low-cost inference endpoints. |
| BASETEN | Production inference platform with autoscaling dedicated deployments. |
| TOGETHER AI | Serverless and dedicated endpoints across open models, LoRA-aware. |
| FIREWORKS AI | Fast serverless inference; multi-LoRA serving on shared base models. |
| MODAL | Serverless Python compute. Run your own vLLM server, full control. |

TRAINING DEEP DIVE

# Specialization beats the frontier.

START TRAINING



POST-TRAINING WITH FLASH